HRGenius: HR Management System (HRMS)

Project Documentation

Program: HCLTech Career Shaper Program

Technology: Java, Spring Boot, Spring Data JPA, REST APIs

Submitted by: Ishan Arora | Roll No.: 2300321530091 | Email Id: ishan.23b1531004@abes.ac.in | College- ABES Engineering College

Project Version: 1.0(Initial Documentation for Review)

1. Introduction

HRGenius is a web-based Human Resource Management System (HRMS) that brings the main HR activities of an organisation into one application. It manages employee records, recruitment and onboarding, payroll, performance reviews and HR analytics through a secure, modular backend built with Java and Spring Boot.

The system exposes its features as REST APIs, so any front end (web or mobile) can use it.

2. Problem Statement

Many organisations still manage HR work using spreadsheets, paper files and separate tools. This causes several problems:

- Employee data is scattered across files and is often duplicated or outdated.
- Recruitment and onboarding are slow because candidate details, interview status and joining formalities are tracked manually.
- Payroll calculation is error-prone and time-consuming, especially for allowances, deductions and tax.
- Performance reviews lack a standard process, so appraisals can be inconsistent.
- HR managers have no quick view of headcount, attrition, salary cost or performance trends, which makes decision-making difficult.

Proposed solution: HRGenius provides one centralised system where HR can manage the complete employee lifecycle, from hiring to performance review, with accurate data, automated payroll and ready-made analytics.

3. Objectives

1. Maintain a single, accurate record of all employees.
2. Simplify and track the recruitment and onboarding process.
3. Automate monthly payroll calculation and payslip generation.
4. Provide a structured way to set goals and review employee performance.
5. Generate HR analytics and reports for management decisions.
6. Provide secure, role-based access to protect sensitive HR data.

4. Scope

In scope

- Employee data management
- Recruitment and onboarding
- Payroll management
- Performance management
- HR analytics and reports
- Authentication and role-based authorisation

Out of scope (for this version)

- Biometric attendance devices
- Mobile application
- Integration with external banking or tax portals

5. Users and Roles

Role	Description	Main Access
Admin	System administrator	Manage users, roles, departments and system settings
HR Manager	HR department staff	Employees, recruitment, payroll, performance, analytics
Manager	Team or department head	View team, give performance reviews, approve goals

Role	Description	Main Access
Employee	Regular staff member	View own profile, payslips and reviews
Candidate	Job applicant	Apply for jobs and track application status

6. Acronyms and Meanings

Acronym	Full Form / Meaning
HRMS	Human Resource Management System
HR	Human Resources
API	Application Programming Interface
REST	Representational State Transfer
JPA	Java Persistence API
ORM	Object Relational Mapping
JDBC	Java Database Connectivity
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
MVC	Model View Controller
JWT	JSON Web Token
RBAC	Role-Based Access Control
DB	Database
SQL	Structured Query Language
JVM	Java Virtual Machine
JDK	Java Development Kit

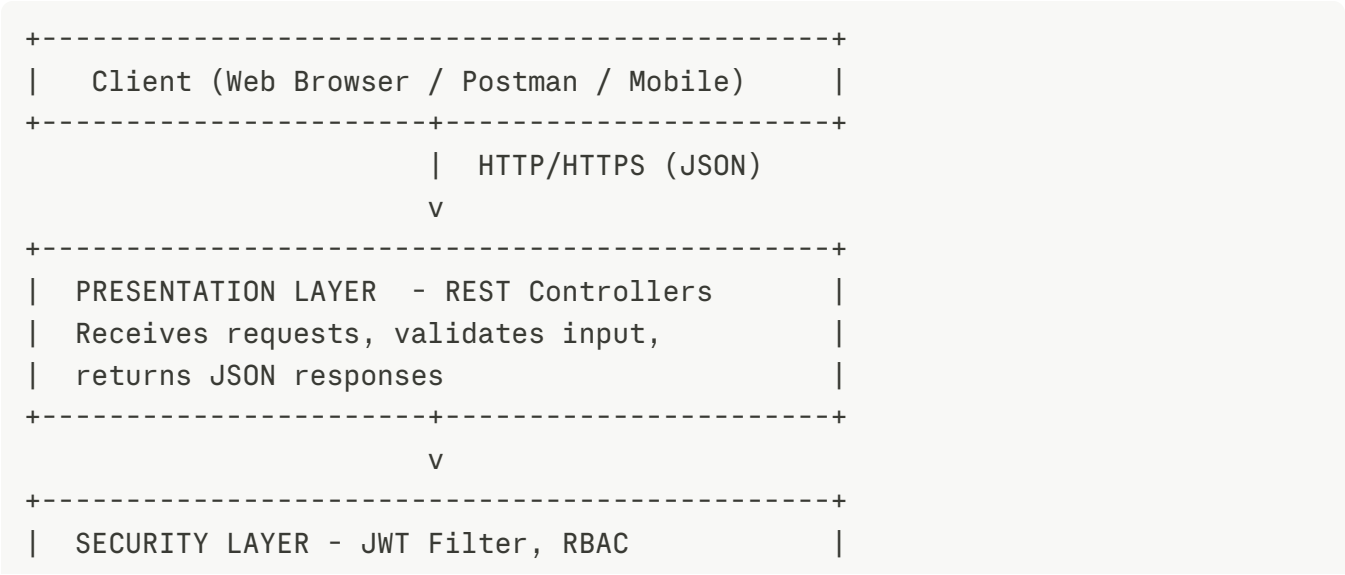
Acronym	Full Form / Meaning
IDE	Integrated Development Environment
ER Diagram	Entity Relationship Diagram
CTC	Cost to Company
PF	Provident Fund
TDS	Tax Deducted at Source
KPI	Key Performance Indicator
SDLC	Software Development Life Cycle
DFD	Data Flow Diagram
PK	Primary Key
FK	Foreign Key

7. Technology Stack

Layer	Technology	Purpose
Language	Java (JDK 17)	Core programming language
Framework	Spring Boot	Rapid development of REST services
Persistence	Spring Data JPA with Hibernate	Maps Java classes to database tables
Security	Spring Security with JWT	Login and role-based access
Database	MySQL (H2 for testing)	Stores all HR data
Build Tool	Maven	Dependency management and build
API Testing	Postman	Testing the REST endpoints
API Documentation	Swagger / OpenAPI	Interactive API documentation
Version Control	Git and GitHub	Source code management
IDE	IntelliJ IDEA / STS	Development

8. System Architecture

HRGenius follows a **layered (3-tier) architecture** in which each layer has one clear responsibility.



```
|  Authenticates user and checks role           |
+-----------------------+-----------------------+
                        v
+-----------------------------------------------+
|  BUSINESS LAYER  - Services                   |
|  Business rules, payroll calculation,         |
|  workflows, analytics logic                   |
+-----------------------+-----------------------+
                        v
+-----------------------------------------------+
|  DATA ACCESS LAYER - JPA Repositories         |
|  CRUD operations through Hibernate ORM        |
+-----------------------+-----------------------+
                        v
+-----------------------------------------------+
|  DATABASE  - MySQL                            |
+-----------------------------------------------+
```

8.1 Layer Responsibilities

Layer	Spring Component	Responsibility
Presentation	`@RestController`	Handles HTTP requests and responses
Security	Spring Security filters	Verifies JWT token and user role
Business	`@Service`	Contains the core logic of the application
Data Access	`@Repository` (JpaRepository)	Talks to the database
Model	`@Entity`	Represents database tables
DTO	Plain Java classes	Carries data between client and server safely

8.2 Request Flow

1. The client sends an HTTP request with a JWT token.
2. The security filter verifies the token and the user's role.
3. The controller receives the request and validates the input.
4. The controller calls the service layer.

5. The service applies business rules and calls the repository.
6. The repository reads or writes data in the database through JPA.
7. The result travels back through the layers and is returned to the client as JSON.

8.3 Project Package Structure

```
com.hrgenius
 ├── controller      (REST controllers)
 ├── service         (business logic)
 ├── repository      (JPA repositories)
 ├── entity          (JPA entities)
 ├── dto             (request/response objects)
 ├── security        (JWT and security config)
 ├── exception       (global exception handling)
 └── config          (application configuration)
```

9. System Design

9.1 Modules

Module 1: Employee Data Management Stores personal, job and department details. HR can add, update, search and deactivate employees. Employees can view their own profile.

Module 2: Recruitment and Onboarding HR posts job openings. Candidates apply, and HR tracks each application through stages: Applied, Shortlisted, Interviewed, Selected or Rejected. Selected candidates move to onboarding, where joining tasks are tracked and the candidate is converted into an employee record.

Module 3: Payroll Management Defines each employee's salary structure (basic pay, allowances, deductions). Monthly payroll is calculated automatically and payslips are generated.

Module 4: Performance Management Managers set goals and review cycles. Employees submit self-assessments, managers give ratings and feedback, and the final rating is stored.

Module 5: HR Analytics Provides summary reports such as headcount by department, hiring trend, attrition rate, payroll cost and performance distribution.

Module 6: Authentication and Authorisation Handles user login, token generation and role-based access to every API.

9.2 Main Entities (Database Design)

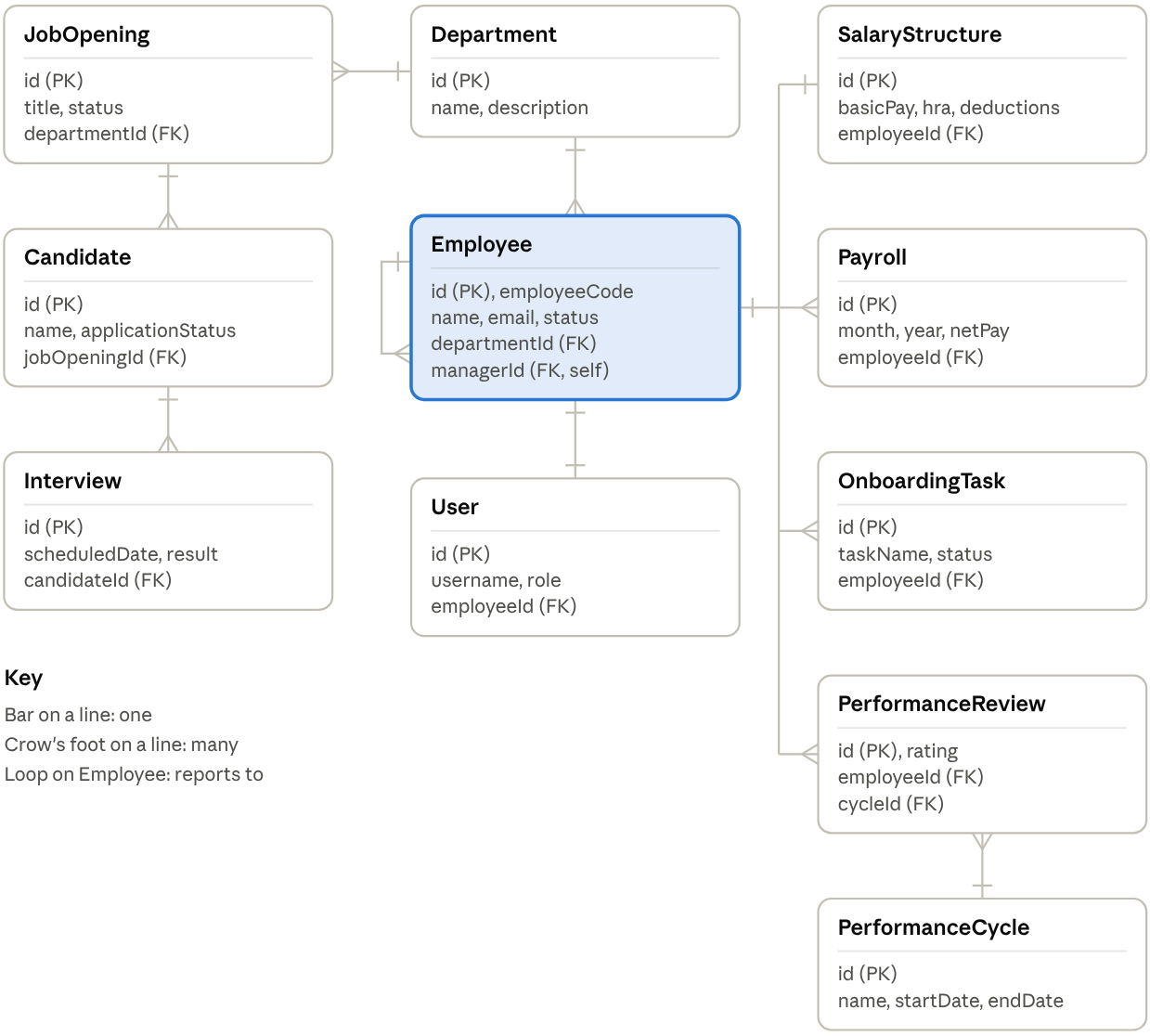
Entity (Table)	Key Fields
User	id, username, password (encrypted), role, status
Employee	id, employeeCode, name, email, phone, dateOfJoining, designation, status, departmentId, managerId
Department	id, name, description
JobOpening	id, title, departmentId, description, vacancies, status, postedDate
Candidate	id, name, email, phone, resume, jobOpeningId, applicationStatus
Interview	id, candidateId, interviewerId, scheduledDate, feedback, result
OnboardingTask	id, employeeId, taskName, status, dueDate
SalaryStructure	id, employeeId, basicPay, hra, allowances, deductions
Payroll	id, employeeId, month, year, grossPay, totalDeductions, netPay, status
PerformanceCycle	id, name, startDate, endDate
PerformanceReview	id, employeeId, reviewerId, cycleId, goals, rating, comments

9.3 Relationships Between Entities (ER Diagram)

- One **Department** has many **Employees** (one-to-many).
- One **Employee** reports to one **Manager**, who is also an Employee (many-to-one, self-reference).
- One **Employee** has one **SalaryStructure** (one-to-one) and many **Payroll** records (one-to-many).
- One **JobOpening** has many **Candidates** (one-to-many).
- One **Candidate** can have many **Interviews** (one-to-many).

- One **Employee** has many **PerformanceReviews** (one-to-many).
- One **PerformanceCycle** has many **PerformanceReviews** (one-to-many).
- One **User** is linked to one **Employee** (one-to-one).

Employee is the hub: every HR module links back to it



ER diagram · 11 entities

Employee is the hub: Department, User, SalaryStructure, Payroll, OnboardingTask and PerformanceReview all link to it, and Employee links to itself through managerId.

9.4 Payroll Calculation Logic

```
Gross Pay        = Basic Pay + HRA + Other Allowances
Total Deductions = PF + Tax (TDS) + Other Deductions
Net Pay          = Gross Pay - Total Deductions
```

9.5 Recruitment Workflow

```
Job Posted -> Candidate Applies -> Shortlisted -> Interview
   -> Selected -> Onboarding Tasks -> Employee Record Created
               \-> Rejected (process ends)
```

9.6 Performance Review Workflow

```
Cycle Created -> Goals Set -> Self Assessment -> Manager Review
   -> Final Rating Saved -> Report Available to Employee and HR
```

10. REST APIs to be Used

All APIs use the base URL `/api/v1`. Data is exchanged in JSON format. All APIs except login and job applications require a valid JWT token.

HTTP methods used: GET (read), POST (create), PUT (update), DELETE (remove).

10.1 Authentication APIs

Method	Endpoint	Description	Access
POST	/auth/login	Log in and receive a JWT token	Public
POST	/auth/register	Create a new user	Admin
POST	/auth/change-password	Change own password	All users

10.2 Employee APIs

Method	Endpoint	Description	Access
POST	/employees	Add a new employee	HR

Method	Endpoint	Description	Access
GET	/employees	List all employees (with search and paging)	HR, Manager
GET	/employees/{id}	Get employee details	HR, Manager, Self
PUT	/employees/{id}	Update employee details	HR
DELETE	/employees/{id}	Deactivate an employee	HR
GET	/employees/me	View own profile	Employee
GET	/employees/{id}/team	View employees reporting to a manager	Manager, HR

10.3 Department APIs

Method	Endpoint	Description	Access
POST	/departments	Create a department	Admin, HR
GET	/departments	List all departments	All users
PUT	/departments/{id}	Update a department	Admin, HR
DELETE	/departments/{id}	Delete a department	Admin

10.4 Recruitment APIs

Method	Endpoint	Description	Access
POST	/jobs	Create a job opening	HR
GET	/jobs	List open job positions	Public
PUT	/jobs/{id}	Update a job opening	HR
POST	/jobs/{id}/apply	Apply for a job	Candidate
GET	/candidates	List candidates (filter by job or status)	HR

Method	Endpoint	Description	Access
PUT	/candidates/{id}/status	Update application status	HR
POST	/interviews	Schedule an interview	HR
PUT	/interviews/{id}/feedback	Submit interview feedback	Interviewer, HR

10.5 Onboarding APIs

Method	Endpoint	Description	Access
POST	/candidates/{id}/convert	Convert a selected candidate to an employee	HR
POST	/onboarding/tasks	Assign an onboarding task	HR
GET	/onboarding/employee/{id}	View onboarding tasks of an employee	HR, Employee
PUT	/onboarding/tasks/{id}	Update task status	HR, Employee

10.6 Payroll APIs

Method	Endpoint	Description	Access
POST	/salary-structures	Define salary structure for an employee	HR
GET	/salary-structures/{employeeId}	View salary structure	HR
POST	/payroll/run	Generate payroll for a month	HR
GET	/payroll/{employeeId}	View payroll history	HR, Self
GET	/payroll/{employeeId}/payslip	Download payslip for a month	HR, Self

10.7 Performance APIs

Method	Endpoint	Description	Access
POST	/performance/cycles	Create a review cycle	HR
GET	/performance/cycles	List review cycles	HR, Manager
POST	/performance/goals	Set goals for an employee	Manager
POST	/performance/reviews	Submit a performance review	Manager
GET	/performance/reviews/{employeeId}	View reviews of an employee	HR, Manager, Self

10.8 Analytics APIs

Method	Endpoint	Description	Access
GET	/analytics/headcount	Employee count by department	HR, Admin
GET	/analytics/hiring	Hiring trend by month	HR, Admin
GET	/analytics/attrition	Attrition rate	HR, Admin
GET	/analytics/payroll-cost	Monthly payroll cost summary	HR, Admin
GET	/analytics/performance	Rating distribution	HR, Admin

10.9 Sample API Request and Response

Request: `POST /api/v1/employees`

```
{
  "name": "Anita Sharma",
  "email": "anita.sharma@example.com",
  "phone": "9876543210",
  "designation": "Software Engineer",
  "departmentId": 2,
  "dateOfJoining": "2026-08-01"
}
```

Response: `201 Created`

```
{
  "id": 101,
  "employeeCode": "EMP101",
  "name": "Anita Sharma",
  "status": "ACTIVE",
  "message": "Employee created successfully"
}
```

10.10 Standard HTTP Status Codes

Code	Meaning	Used When
200	OK	Request successful
201	Created	New record created
400	Bad Request	Invalid input data
401	Unauthorized	Missing or invalid token
403	Forbidden	Role does not have permission
404	Not Found	Record does not exist
500	Internal Server Error	Unexpected server problem

11. Security Design

- Passwords are stored in encrypted form using BCrypt hashing.
- Login returns a JWT token, which must be sent with every protected request.
- Role-Based Access Control (RBAC) restricts each API to the roles listed above.
- Input is validated using Bean Validation (for example `@NotNull`, `@Email`).
- A global exception handler returns clear error messages without exposing internal details.
- Salary and personal data are visible only to authorised roles.

12. Non-Functional Requirements

Requirement	Description
Performance	Common requests should respond within 2 seconds
Security	Authentication, authorisation and encrypted passwords
Scalability	Layered design allows new modules to be added easily
Maintainability	Clean package structure and separation of concerns
Reliability	Database transactions keep payroll and employee data consistent
Usability	Consistent, well-documented REST APIs

13. Future Enhancements

- Attendance and leave management
- Email notifications for interviews, payslips and reviews
- Front end dashboard (React or Angular)
- Resume parsing and automatic candidate ranking
- Mobile application
- Export of reports to PDF and Excel

14. Conclusion

HRGenius provides a simple, secure and well-structured solution for managing the core HR functions of an organisation. By using Spring Boot, JPA and REST APIs in a layered architecture, the system stays easy to understand, maintain and extend. It reduces manual work, improves data accuracy and gives HR managers useful insights for better decisions.

15. References

- Spring Boot Documentation: https://spring.io/projects/spring-boot
- Spring Data JPA Documentation: https://spring.io/projects/spring-data-jpa
- Spring Security Documentation: https://spring.io/projects/spring-security
- Hibernate ORM Documentation: https://hibernate.org/orm/
- REST API Design Guidelines: https://restfulapi.net